Mnemonic Similarity Task (MST) – Descriptive Statistics Analysis

Sourav Nath, Piyush Agarwal, Rajarshi Chatterjee

2026-03-10

Contents

1. Setup and Data Loading **2**

1.1 Define Helper Functions 2

1.2 Load Data from All Three Conditions 3

1.3 Load Similarity Bins 4

2. Compute Per-Participant Summary Metrics **5**

2.1 Summary Statistics Table 6

3. Response Proportion Analysis **6**

3.1 Figure 1: Response Proportions by Stimulus Type 7

4. Key Metrics Comparison Across Conditions **8**

4.1 Figure 2: LDI and REC Across Conditions 8

4.2 Figure 3: Hit Rate and False Alarm Rate 9

5. Distribution Analysis **10**

5.1 Figure 4: Distribution of LDI Scores 10

5.2 Normal Distribution and t-Distribution Fitting 11

5.3 One-Sample t-Tests (Descriptive) 13

6. Reaction Time Analysis **15**

6.1 Figure 6: Reaction Times by Stimulus Type 15

6.2 Figure 7: Mean RT by Response Type 16

7. Similarity Bin Analysis **17**

7.1 Figure 8: Lure Performance by Similarity Bin 18

8. Lure Position Analysis (pre, mid, post) **19**

8.1 Figure 10: Lure Performance by Position 20

9. Additional Descriptive Statistics **21**

9.1 Overall Accuracy 21

9.2 Participant-Level Variability 22

10. Summary of Key Findings **23**

1. Setup and Data Loading

```
# Load required libraries
library(ggplot2)
library(dplyr)
library(tidyr)
library(knitr)

# Clear workspace
rm(list = setdiff(ls(), lsf.str()))
```

1.1 Define Helper Functions

```
# Function to read all test files from a condition folder
read_test_files <- function(data_dir, condition_name) {
  # List all test CSV files
  test_files <- list.files(data_dir, pattern = "MST_test.*\\.csv$", full.names = TRUE)

  all_data <- data.frame()

  for (f in test_files) {
    # Extract participant ID from filename
    participant_id <- sub("^(\\d+)_MST_test.*", "\\1", basename(f))

    df <- read.csv(f, stringsAsFactors = FALSE)

    # Keep only trial rows (rows with actual image_path and response)
    # In test files, trial rows have non-empty key_resp_3.keys and image_path
    trial_cols <- c("thisN", "key_resp_3.keys", "key_resp_3.rt",
                    "image_path", "position_of_stimuli")

    # Check which columns exist
    existing_cols <- trial_cols[trial_cols %in% names(df)]
    df_sub <- df[existing_cols]

    # Filter to actual trial rows
    df_sub <- df_sub[!is.na(df_sub$image_path) & df_sub$image_path != "" &
                     !is.na(df_sub$key_resp_3.keys) & df_sub$key_resp_3.keys != "", ]

    if (nrow(df_sub) > 0) {
      df_sub$participant <- participant_id
      df_sub$condition <- condition_name
```

```
      all_data <- rbind(all_data, df_sub)
    }
  }

  return(all_data)
}

# Function to read bins file
read_bins <- function(filepath) {
  bins <- read.delim(filepath, header = FALSE, sep = "\t",
                     col.names = c("item_num", "bin"))
  return(bins)
}
```

1.2 Load Data from All Three Conditions

```
base_dir <- "MST_Data"

# Load test data from all conditions
item_only_data <- read_test_files(
  file.path(base_dir, "item_only", "item_only_data"), "Item Only")
task_only_data <- read_test_files(
  file.path(base_dir, "task_only", "task_only_data"), "Task Only")
both_data <- read_test_files(
  file.path(base_dir, "Both_item_task", "both_data"), "Both")

# Combine all data
all_test_data <- rbind(item_only_data, task_only_data, both_data)

# Convert response to factor
all_test_data$response <- factor(all_test_data$key_resp_3.keys,
                                 levels = c("o", "s", "n"),
                                 labels = c("Old", "Similar", "New"))

# Convert reaction time to numeric
all_test_data$rt <- as.numeric(all_test_data$key_resp_3.rt)

# Classify stimulus type based on position_of_stimuli
# pre, mid, post = Lure (similar items at different positions)
# none = Foil (completely new)
# For targets: items shown at encoding that appear again exactly
# In this dataset: 'post' and 'pre' and 'mid' positions of lures
# The position_of_stimuli indicates the type:
#   - Items from Objects/Scenes folders with 'a' suffix shown at encoding = targets if shown again
#   - Items with 'b' suffix = lures (similar versions)
#   - Foils = brand new items from Foils folder

all_test_data$stimulus_type <- ifelse(
  grepl("^Foils", all_test_data$image_path), "Foil",
  ifelse(grepl("b\\.", all_test_data$image_path), "Lure", "Target")
)
```

```r
all_test_data$stimulus_type <- factor(all_test_data$stimulus_type,
                                      levels = c("Target", "Lure", "Foil"))

# Convert condition to factor
all_test_data$condition <- factor(all_test_data$condition,
                                  levels = c("Item Only", "Task Only", "Both"))

cat("Total trials loaded:", nrow(all_test_data), "\n")
```

```
## Total trials loaded: 24150
```

```r
cat("Participants per condition:\n")
```

```
## Participants per condition:
```

```r
print(tapply(all_test_data$participant, all_test_data$condition,
             function(x) length(unique(x))))
```

```
## Item Only Task Only      Both
##        56        53        51
```

```r
cat("\nTrials per stimulus type:\n")
```

```
##
## Trials per stimulus type:
```

```r
print(table(all_test_data$stimulus_type))
```

```
##
## Target   Lure   Foil
##   9660   9660   4830
```

1.3 Load Similarity Bins

```r
# Load bins for Set6 (Objects)
bins_set6 <- read_bins(file.path(base_dir, "item_only", "Set6 bins.txt"))
bins_set6$set <- "Objects"

# Load bins for SetScC (Scenes)
bins_scc <- read_bins(file.path(base_dir, "item_only", "SetScC bins.txt"))
bins_scc$set <- "Scenes"

# Combine bins
all_bins <- rbind(bins_set6, bins_scc)
cat("Bins distribution:\n")
```

```
## Bins distribution:
```

```r
print(table(all_bins$bin))
```

```
##
##  1  2  3  4  5
## 78 76 78 76 76
```

2. Compute Per-Participant Summary Metrics

```r
# For each participant, compute key metrics
participant_metrics <- all_test_data %>%
  group_by(participant, condition) %>%
  summarise(
    # Total trials by stimulus type
    n_targets = sum(stimulus_type == "Target", na.rm = TRUE),
    n_lures = sum(stimulus_type == "Lure", na.rm = TRUE),
    n_foils = sum(stimulus_type == "Foil", na.rm = TRUE),

    # Hit Rate: P("Old" | Target)
    hit_rate = sum(stimulus_type == "Target" & response == "Old", na.rm = TRUE) /
               sum(stimulus_type == "Target", na.rm = TRUE),

    # False Alarm Rate: P("Old" | Foil)
    fa_rate = sum(stimulus_type == "Foil" & response == "Old", na.rm = TRUE) /
              sum(stimulus_type == "Foil", na.rm = TRUE),

    # Correct Rejection Rate: P("New" | Foil)
    cr_rate = sum(stimulus_type == "Foil" & response == "New", na.rm = TRUE) /
              sum(stimulus_type == "Foil", na.rm = TRUE),

    # Lure Discrimination Index (LDI): P("Similar" | Lure) - P("Similar" | Foil)
    p_sim_lure = sum(stimulus_type == "Lure" & response == "Similar", na.rm = TRUE) /
                 sum(stimulus_type == "Lure", na.rm = TRUE),
    p_sim_foil = sum(stimulus_type == "Foil" & response == "Similar", na.rm = TRUE) /
                 sum(stimulus_type == "Foil", na.rm = TRUE),
    LDI = p_sim_lure - p_sim_foil,

    # Recognition Memory (REC): P("Old" | Target) - P("Old" | Foil)
    REC = hit_rate - fa_rate,

    # Mean Reaction Times
    rt_targets = mean(rt[stimulus_type == "Target"], na.rm = TRUE),
    rt_lures = mean(rt[stimulus_type == "Lure"], na.rm = TRUE),
    rt_foils = mean(rt[stimulus_type == "Foil"], na.rm = TRUE),
    rt_overall = mean(rt, na.rm = TRUE),

    .groups = "drop"
  )

cat("Participant metrics computed for", nrow(participant_metrics), "participants\n")
```

```
## Participant metrics computed for 160 participants
```

2.1 Summary Statistics Table

```r
# Compute group-level summary statistics
summary_stats <- participant_metrics %>%
  group_by(condition) %>%
  summarise(
    N = n(),
    `Hit Rate (M)` = round(mean(hit_rate, na.rm = TRUE), 3),
    `Hit Rate (SD)` = round(sd(hit_rate, na.rm = TRUE), 3),
    `FA Rate (M)` = round(mean(fa_rate, na.rm = TRUE), 3),
    `FA Rate (SD)` = round(sd(fa_rate, na.rm = TRUE), 3),
    `LDI (M)` = round(mean(LDI, na.rm = TRUE), 3),
    `LDI (SD)` = round(sd(LDI, na.rm = TRUE), 3),
    `REC (M)` = round(mean(REC, na.rm = TRUE), 3),
    `REC (SD)` = round(sd(REC, na.rm = TRUE), 3),
    `CR Rate (M)` = round(mean(cr_rate, na.rm = TRUE), 3),
    `CR Rate (SD)` = round(sd(cr_rate, na.rm = TRUE), 3),
    .groups = "drop"
  )

kable(summary_stats, caption = "Table 1: Summary Statistics by Condition (Mean ± SD)")
```

Table 1: Table 1: Summary Statistics by Condition (Mean ± SD)

condition	N	Hit Rate (M)	Hit Rate (SD)	FA Rate (M)	FA Rate (SD)	LDI (M)	LDI (SD)	REC (M)	REC (SD)	CR Rate (M)	CR Rate (SD)
Item Only	56	0.669	0.145	0.035	0.059	0.319	0.169	0.634	0.153	0.775	0.128
Task Only	53	0.627	0.168	0.031	0.050	0.381	0.188	0.596	0.174	0.775	0.136
Both	51	0.642	0.133	0.047	0.071	0.269	0.175	0.594	0.139	0.755	0.117

3. Response Proportion Analysis

```r
# Compute response proportions for each stimulus type × condition
response_props <- all_test_data %>%
  group_by(condition, stimulus_type, response) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(condition, stimulus_type) %>%
  mutate(proportion = count / sum(count)) %>%
  ungroup()

# Display as table
response_table <- response_props %>%
  select(condition, stimulus_type, response, proportion) %>%
  pivot_wider(names_from = response, values_from = proportion, values_fill = 0) %>%
  mutate(across(where(is.numeric), ~ round(., 3)))
```

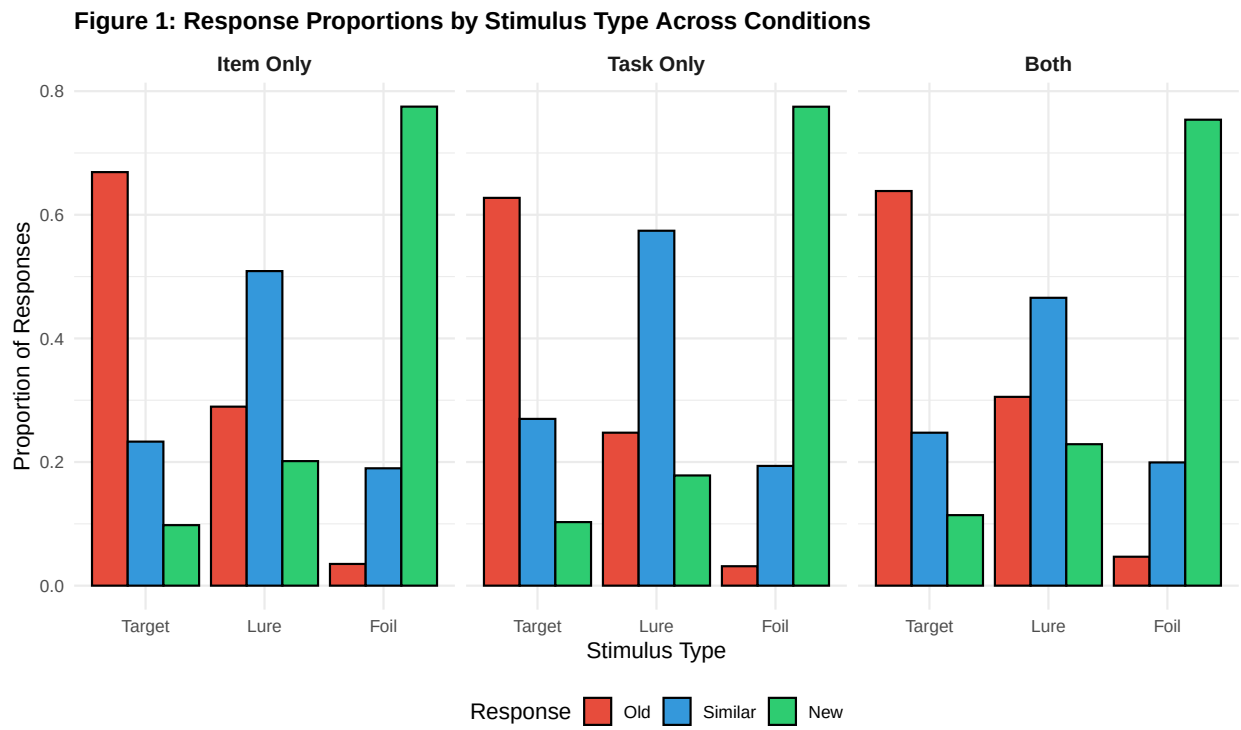
```r
kable(response_table, caption = "Table 2: Response Proportions by Stimulus Type and Condition")
```

Table 2: Table 2: Response Proportions by Stimulus Type and Condition

condition	stimulus_type	Old	Similar	New
Item Only	Target	0.669	0.233	0.098
Item Only	Lure	0.290	0.509	0.201
Item Only	Foil	0.035	0.190	0.775
Task Only	Target	0.627	0.270	0.103
Task Only	Lure	0.247	0.574	0.178
Task Only	Foil	0.031	0.194	0.775
Both	Target	0.638	0.247	0.114
Both	Lure	0.305	0.466	0.229
Both	Foil	0.047	0.199	0.754

3.1 Figure 1: Response Proportions by Stimulus Type

```r
ggplot(response_props, aes(x = stimulus_type, y = proportion, fill = response)) +
  geom_bar(stat = "identity", position = "dodge", color = "black", size = 0.3) +
  facet_wrap(~ condition) +
  scale_fill_manual(values = c("Old" = "#E74C3C", "Similar" = "#3498DB", "New" = "#2ECC71"),
                    name = "Response") +
  labs(title = "Figure 1: Response Proportions by Stimulus Type Across Conditions",
       x = "Stimulus Type", y = "Proportion of Responses") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 12),
        legend.position = "bottom")
```



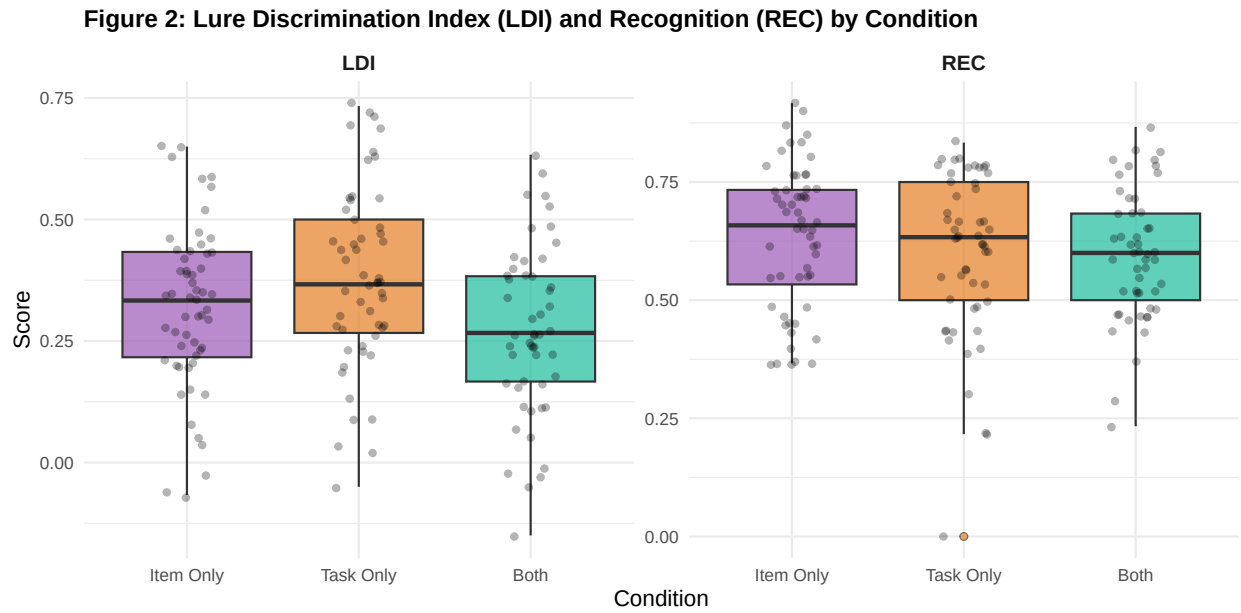

```
ggsave("figures/fig1_response_proportions.png", width = 10, height = 6, dpi = 300)
```

4. Key Metrics Comparison Across Conditions

4.1 Figure 2: LDI and REC Across Conditions

```
# Prepare data for plotting
metrics_long <- participant_metrics %>%
  select(participant, condition, LDI, REC, hit_rate, fa_rate, cr_rate) %>%
  pivot_longer(cols = c(LDI, REC, hit_rate, fa_rate, cr_rate),
               names_to = "metric", values_to = "value")

# LDI and REC boxplots
ldi_rec_data <- metrics_long %>%
  filter(metric %in% c("LDI", "REC"))

ggplot(ldi_rec_data, aes(x = condition, y = value, fill = condition)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21) +
  geom_jitter(width = 0.15, alpha = 0.3, size = 1.5) +
  facet_wrap(~ metric, scales = "free_y") +
  scale_fill_manual(values = c("Item Only" = "#9B59B6",
                               "Task Only" = "#E67E22",
                               "Both" = "#1ABC9C")) +
  labs(title = "Figure 2: Lure Discrimination Index (LDI) and Recognition (REC) by Condition",
       x = "Condition", y = "Score") +
  theme_minimal(base_size = 13) +
```

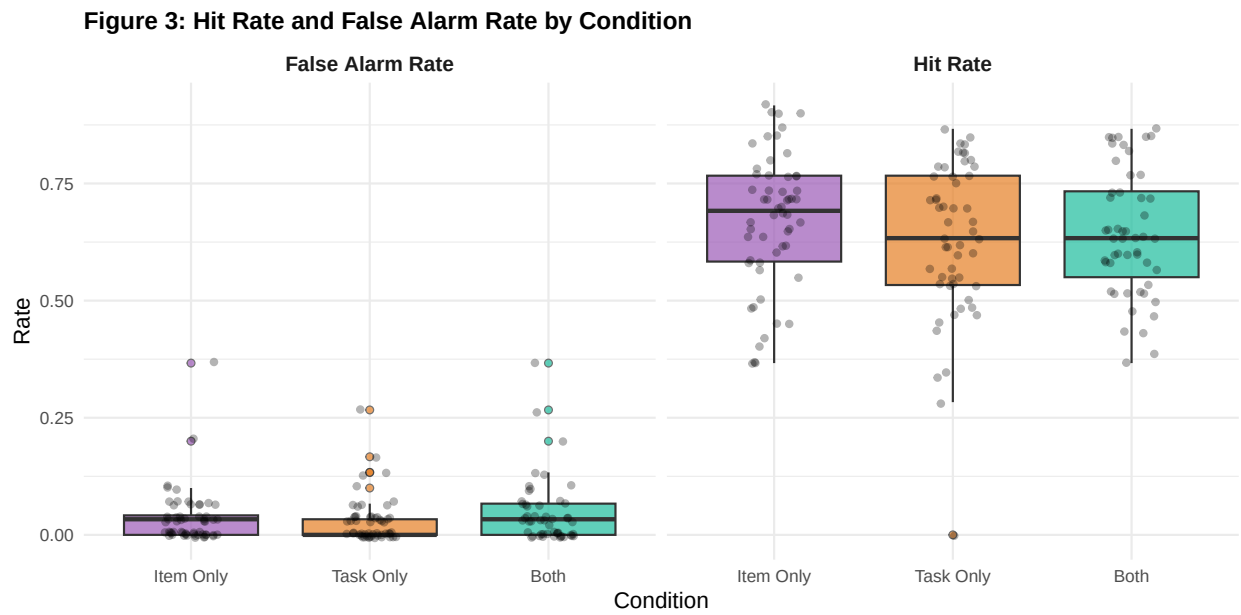
```
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 12),
        legend.position = "none")
```




```
ggsave("figures/fig2_ldi_rec_boxplot.png", width = 10, height = 5, dpi = 300)
```

4.2 Figure 3: Hit Rate and False Alarm Rate

```
hit_fa_data <- metrics_long %>%
  filter(metric %in% c("hit_rate", "fa_rate")) %>%
  mutate(metric = recode(metric,
                         "hit_rate" = "Hit Rate",
                         "fa_rate" = "False Alarm Rate"))

ggplot(hit_fa_data, aes(x = condition, y = value, fill = condition)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21) +
  geom_jitter(width = 0.15, alpha = 0.3, size = 1.5) +
  facet_wrap(~ metric) +
  scale_fill_manual(values = c("Item Only" = "#9B59B6",
                               "Task Only" = "#E67E22",
                               "Both" = "#1ABC9C")) +
  labs(title = "Figure 3: Hit Rate and False Alarm Rate by Condition",
       x = "Condition", y = "Rate") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 12),
        legend.position = "none")
```

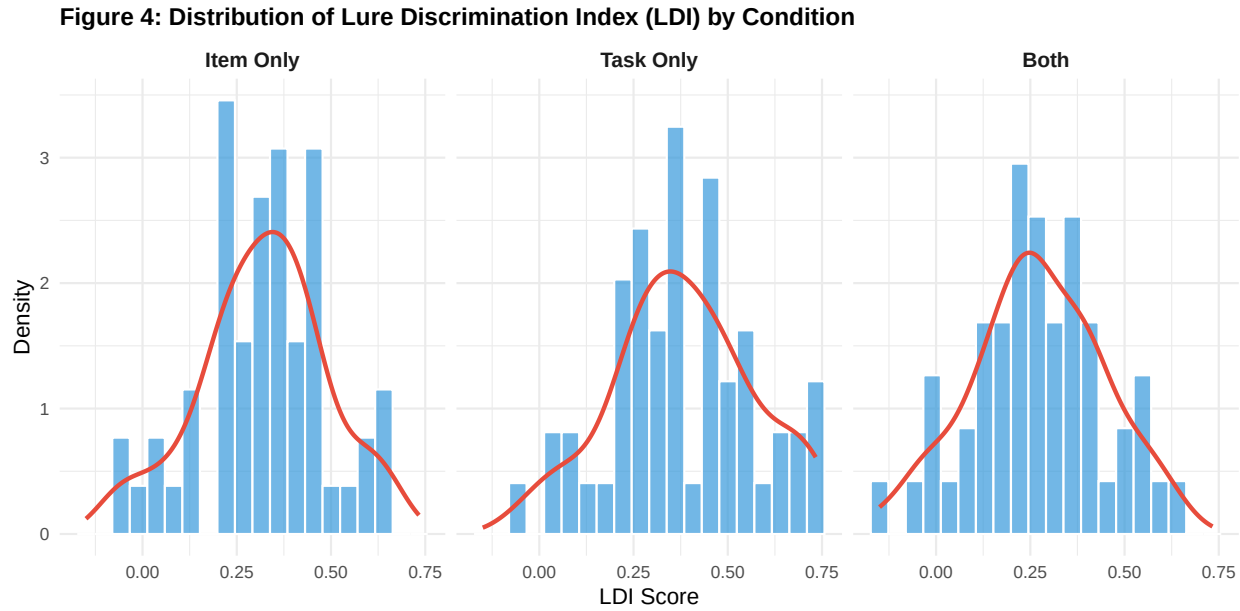



```
ggsave("figures/fig3_hit_fa_boxplot.png", width = 10, height = 5, dpi = 300)
```

5. Distribution Analysis

5.1 Figure 4: Distribution of LDI Scores

```
# Histogram with overlaid normal curve
ggplot(participant_metrics, aes(x = LDI)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20,
                 fill = "#3498DB", color = "white", alpha = 0.7) +
  geom_density(color = "#E74C3C", linewidth = 1.2) +
  facet_wrap(~ condition) +
  labs(title = "Figure 4: Distribution of Lure Discrimination Index (LDI) by Condition",
       x = "LDI Score", y = "Density") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 12))
```




```r
ggsave("figures/fig4_ldi_distribution.png", width = 10, height = 5, dpi = 300)
```

5.2 Normal Distribution and t-Distribution Fitting

```r
# For each condition, compare LDI distribution to normal and t-distribution
cat("=== Shapiro-Wilk Normality Tests for LDI ===\n\n")
```

```
## === Shapiro-Wilk Normality Tests for LDI ===
```

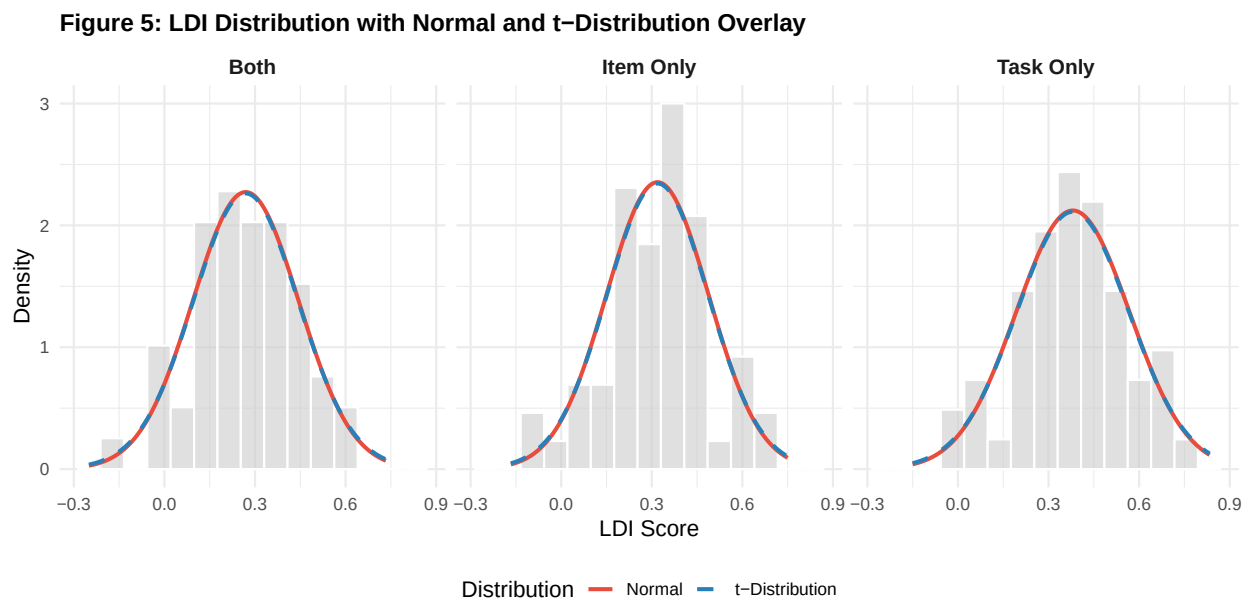
```r
for (cond in levels(participant_metrics$condition)) {
  ldi_vals <- participant_metrics$LDI[participant_metrics$condition == cond]
  ldi_vals <- ldi_vals[!is.na(ldi_vals)]

  cat(paste0("Condition: ", cond, "\n"))
  cat(paste0("  N = ", length(ldi_vals), "\n"))
  cat(paste0("  Mean = ", round(mean(ldi_vals), 4), "\n"))
  cat(paste0("  SD = ", round(sd(ldi_vals), 4), "\n"))
  cat(paste0("  Skewness = ", round(moments::skewness(ldi_vals), 4), "\n"))
  cat(paste0("  Kurtosis = ", round(moments::kurtosis(ldi_vals), 4), "\n"))

  sw_test <- shapiro.test(ldi_vals)
  cat(paste0("  Shapiro-Wilk W = ", round(sw_test$statistic, 4),
             ", p = ", round(sw_test$p.value, 4), "\n\n"))
}
```

```
## Condition: Item Only
##   N = 56
##   Mean = 0.319
##   SD = 0.1694
##   Skewness = -0.2225
```

```
##   Kurtosis = 2.9319
##   Shapiro-Wilk W = 0.9788, p = 0.4278
##
## Condition: Task Only
##   N = 53
##   Mean = 0.3805
##   SD = 0.188
##   Skewness = -0.0604
##   Kurtosis = 2.6088
##   Shapiro-Wilk W = 0.9829, p = 0.6441
##
## Condition: Both
##   N = 51
##   Mean = 0.2693
##   SD = 0.1755
##   Skewness = -0.1087
##   Kurtosis = 2.6836
##   Shapiro-Wilk W = 0.9893, p = 0.9246
```

```r
# Compare Normal vs t-distribution fit for LDI scores
plot_data <- data.frame()

for (cond in levels(participant_metrics$condition)) {
  ldi_vals <- participant_metrics$LDI[participant_metrics$condition == cond]
  ldi_vals <- ldi_vals[!is.na(ldi_vals)]

  m <- mean(ldi_vals)
  s <- sd(ldi_vals)
  n <- length(ldi_vals)

  x_seq <- seq(min(ldi_vals) - 0.1, max(ldi_vals) + 0.1, length.out = 200)

  # Normal density
  normal_density <- dnorm(x_seq, mean = m, sd = s)

  # t-distribution density (scaled) with n-1 degrees of freedom
  t_density <- dt((x_seq - m) / s, df = n - 1) / s

  temp <- data.frame(
    x = rep(x_seq, 2),
    density = c(normal_density, t_density),
    distribution = rep(c("Normal", "t-Distribution"), each = length(x_seq)),
    condition = cond
  )
  plot_data <- rbind(plot_data, temp)
}

ggplot() +
  geom_histogram(data = participant_metrics,
                 aes(x = LDI, y = after_stat(density)),
                 bins = 15, fill = "grey80", color = "white", alpha = 0.6) +
  geom_line(data = plot_data,
            aes(x = x, y = density, color = distribution, linetype = distribution),
            linewidth = 1.1) +
```

```r
  facet_wrap(~ condition) +
  scale_color_manual(values = c("Normal" = "#E74C3C", "t-Distribution" = "#2980B9"),
                     name = "Distribution") +
  scale_linetype_manual(values = c("Normal" = "solid", "t-Distribution" = "dashed"),
                        name = "Distribution") +
  labs(title = "Figure 5: LDI Distribution with Normal and t-Distribution Overlay",
       x = "LDI Score", y = "Density") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 12),
        legend.position = "bottom")
```




```r
ggsave("figures/fig5_normal_t_comparison.png", width = 10, height = 5, dpi = 300)
```

5.3 One-Sample t-Tests (Descriptive)

```r
# Test whether LDI is significantly different from 0 (chance) for each condition
cat("=== One-Sample t-Tests: Is LDI significantly > 0? ===\n\n")
```

```
## === One-Sample t-Tests: Is LDI significantly > 0? ===
```

```r
for (cond in levels(participant_metrics$condition)) {
  ldi_vals <- participant_metrics$LDI[participant_metrics$condition == cond]
  ldi_vals <- ldi_vals[!is.na(ldi_vals)]

  t_result <- t.test(ldi_vals, mu = 0, alternative = "greater")

  cat(paste0("Condition: ", cond, "\n"))
  cat(paste0("  Mean LDI = ", round(t_result$estimate, 4), "\n"))
```

```r
  cat(paste0("  t(", t_result$parameter, ") = ", round(t_result$statistic, 4), "\n"))
  cat(paste0("  p-value = ", format(t_result$p.value, scientific = TRUE, digits = 4), "\n"))
  cat(paste0("  95% CI: [", round(t_result$conf.int[1], 4), ", Inf]\n"))

  # Cohen's d
  cohens_d <- mean(ldi_vals) / sd(ldi_vals)
  cat(paste0("  Cohen's d = ", round(cohens_d, 4), "\n\n"))
}
```

```
## Condition: Item Only
##   Mean LDI = 0.319
##   t(55) = 14.0904
##   p-value = 3.377e-20
##   95% CI: [0.2812, Inf]
##   Cohen's d = 1.8829
##
## Condition: Task Only
##   Mean LDI = 0.3805
##   t(52) = 14.7356
##   p-value = 1.668e-20
##   95% CI: [0.3373, Inf]
##   Cohen's d = 2.0241
##
## Condition: Both
##   Mean LDI = 0.2693
##   t(50) = 10.9588
##   p-value = 3.375e-15
##   95% CI: [0.2281, Inf]
##   Cohen's d = 1.5345
```

```r
# Same for REC
cat("=== One-Sample t-Tests: Is REC significantly > 0? ===\n\n")
```

```
## === One-Sample t-Tests: Is REC significantly > 0? ===
```

```r
for (cond in levels(participant_metrics$condition)) {
  rec_vals <- participant_metrics$REC[participant_metrics$condition == cond]
  rec_vals <- rec_vals[!is.na(rec_vals)]

  t_result <- t.test(rec_vals, mu = 0, alternative = "greater")

  cat(paste0("Condition: ", cond, "\n"))
  cat(paste0("  Mean REC = ", round(t_result$estimate, 4), "\n"))
  cat(paste0("  t(", t_result$parameter, ") = ", round(t_result$statistic, 4), "\n"))
  cat(paste0("  p-value = ", format(t_result$p.value, scientific = TRUE, digits = 4), "\n"))
  cat(paste0("  95% CI: [", round(t_result$conf.int[1], 4), ", Inf]\n\n"))
}
```

```
## Condition: Item Only
##   Mean REC = 0.6339
##   t(55) = 30.9626
##   p-value = 8.672e-37
```

```
##   95% CI: [0.5997, Inf]
##
## Condition: Task Only
##   Mean REC = 0.5959
##   t(52) = 24.8798
##   p-value = 7.566e-31
##   95% CI: [0.5558, Inf]
##
## Condition: Both
##   Mean REC = 0.5943
##   t(50) = 30.5261
##   p-value = 2.713e-34
##   95% CI: [0.5617, Inf]
```

6. Reaction Time Analysis

```r
# RT summary statistics
rt_summary <- all_test_data %>%
  filter(!is.na(rt) & rt > 0) %>%
  group_by(condition, stimulus_type) %>%
  summarise(
    Mean_RT = round(mean(rt, na.rm = TRUE), 3),
    SD_RT = round(sd(rt, na.rm = TRUE), 3),
    Median_RT = round(median(rt, na.rm = TRUE), 3),
    N = n(),
    .groups = "drop"
  )

kable(rt_summary, caption = "Table 3: Reaction Time (seconds) by Condition and Stimulus Type")
```

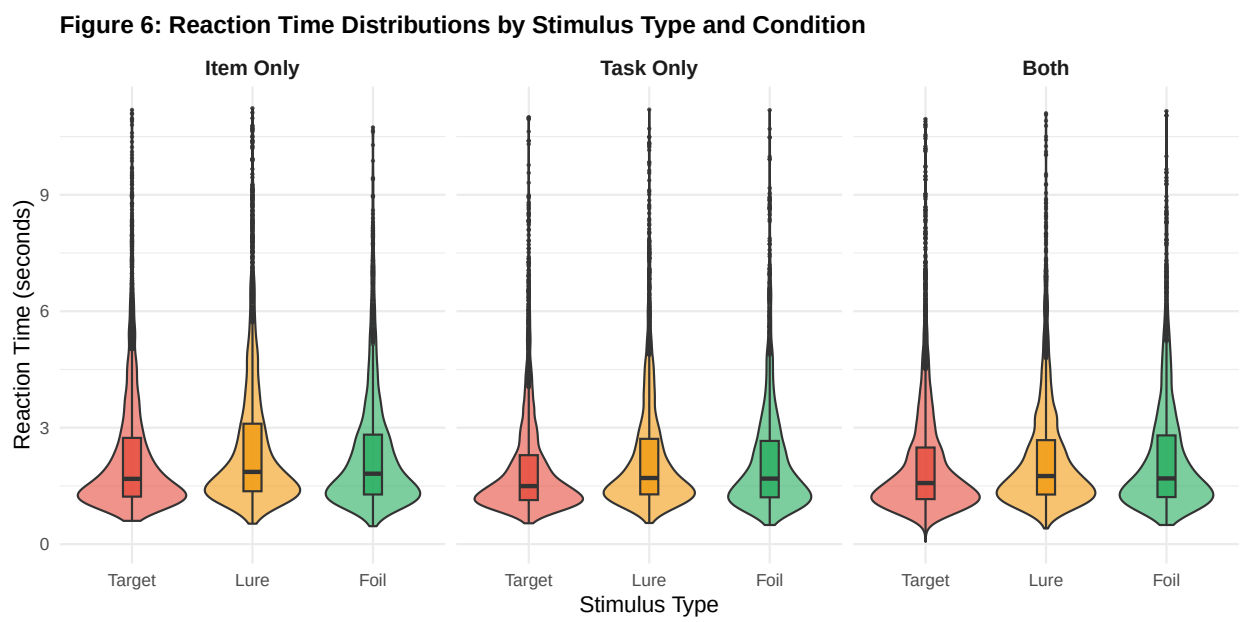
Table 3: Table 3: Reaction Time (seconds) by Condition and Stimulus Type

condition	stimulus_type	Mean_RT	SD_RT	Median_RT	N
Item Only	Target	2.520	2.714	1.698	3360
Item Only	Lure	2.831	3.040	1.889	3360
Item Only	Foil	2.643	3.190	1.836	1680
Task Only	Target	2.096	2.133	1.499	3180
Task Only	Lure	2.338	1.901	1.708	3180
Task Only	Foil	2.278	1.822	1.691	1590
Both	Target	2.192	2.142	1.578	3120
Both	Lure	2.388	2.264	1.765	3120
Both	Foil	2.426	2.320	1.708	1560

6.1 Figure 6: Reaction Times by Stimulus Type

```r
rt_data <- all_test_data %>%
  filter(!is.na(rt) & rt > 0 & rt < quantile(rt, 0.99, na.rm = TRUE))
```

```r
ggplot(rt_data, aes(x = stimulus_type, y = rt, fill = stimulus_type)) +
  geom_violin(alpha = 0.6, trim = TRUE) +
  geom_boxplot(width = 0.15, alpha = 0.8, outlier.size = 0.5) +
  facet_wrap(~ condition) +
  scale_fill_manual(values = c("Target" = "#E74C3C",
                               "Lure" = "#F39C12",
                               "Foil" = "#27AE60")) +
  labs(title = "Figure 6: Reaction Time Distributions by Stimulus Type and Condition",
       x = "Stimulus Type", y = "Reaction Time (seconds)") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 12),
        legend.position = "none")
```

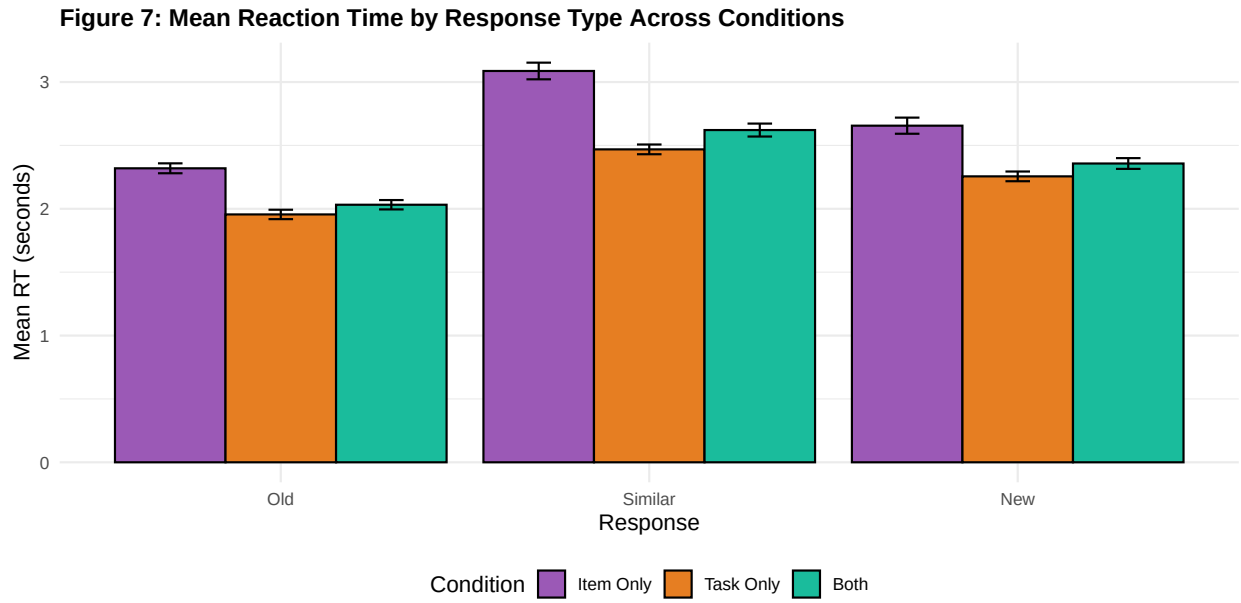



```r
ggsave("figures/fig6_rt_violin.png", width = 10, height = 5, dpi = 300)
```

6.2 Figure 7: Mean RT by Response Type

```r
rt_by_response <- all_test_data %>%
  filter(!is.na(rt) & rt > 0 & !is.na(response)) %>%
  group_by(condition, response) %>%
  summarise(
    mean_rt = mean(rt, na.rm = TRUE),
    se_rt = sd(rt, na.rm = TRUE) / sqrt(n()),
    .groups = "drop"
  )

ggplot(rt_by_response, aes(x = response, y = mean_rt, fill = condition)) +
  geom_bar(stat = "identity", position = "dodge", color = "black", size = 0.3) +
  geom_errorbar(aes(ymin = mean_rt - se_rt, ymax = mean_rt + se_rt),
```

```
                 position = position_dodge(0.9), width = 0.2) +
  scale_fill_manual(values = c("Item Only" = "#9B59B6",
                               "Task Only" = "#E67E22",
                               "Both" = "#1ABC9C")) +
  labs(title = "Figure 7: Mean Reaction Time by Response Type Across Conditions",
       x = "Response", y = "Mean RT (seconds)",
       fill = "Condition") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        legend.position = "bottom")
```




```
ggsave("figures/fig7_rt_by_response.png", width = 10, height = 5, dpi = 300)
```

7. Similarity Bin Analysis

```
# Extract item number from image path for lure items
# Lure items have 'b' suffix: e.g., Objects\039b.jpg -> item 39
lure_data <- all_test_data %>%
  filter(stimulus_type == "Lure") %>%
  mutate(
    # Extract the number from the image path
    item_num_str = sub(".*[/\\\\](\\d+)b\\.jpg$", "\\1", image_path),
    item_num = as.integer(item_num_str),
    # Determine if object or scene
    stim_set = ifelse(grepl("^Objects", image_path), "Objects", "Scenes")
  ) %>%
  filter(!is.na(item_num))

# Merge with bins
# For Objects: use bins_set6, for Scenes: use bins_scc
```

```r
lure_with_bins <- lure_data %>%
  left_join(
    rbind(
      bins_set6 %>% mutate(stim_set = "Objects"),
      bins_scc %>% mutate(stim_set = "Scenes")
    ),
    by = c("item_num" = "item_num", "stim_set" = "stim_set")
  ) %>%
  filter(!is.na(bin))

# Compute LDI-like metric by bin
bin_performance <- lure_with_bins %>%
  group_by(condition, bin) %>%
  summarise(
    n_trials = n(),
    p_old = mean(response == "Old", na.rm = TRUE),
    p_similar = mean(response == "Similar", na.rm = TRUE),
    p_new = mean(response == "New", na.rm = TRUE),
    mean_rt = mean(rt, na.rm = TRUE),
    .groups = "drop"
  )

# Table
kable(bin_performance %>% mutate(across(where(is.numeric), ~ round(., 3))),
      caption = "Table 4: Lure Performance by Similarity Bin (1=most similar, 5=least similar)")
```

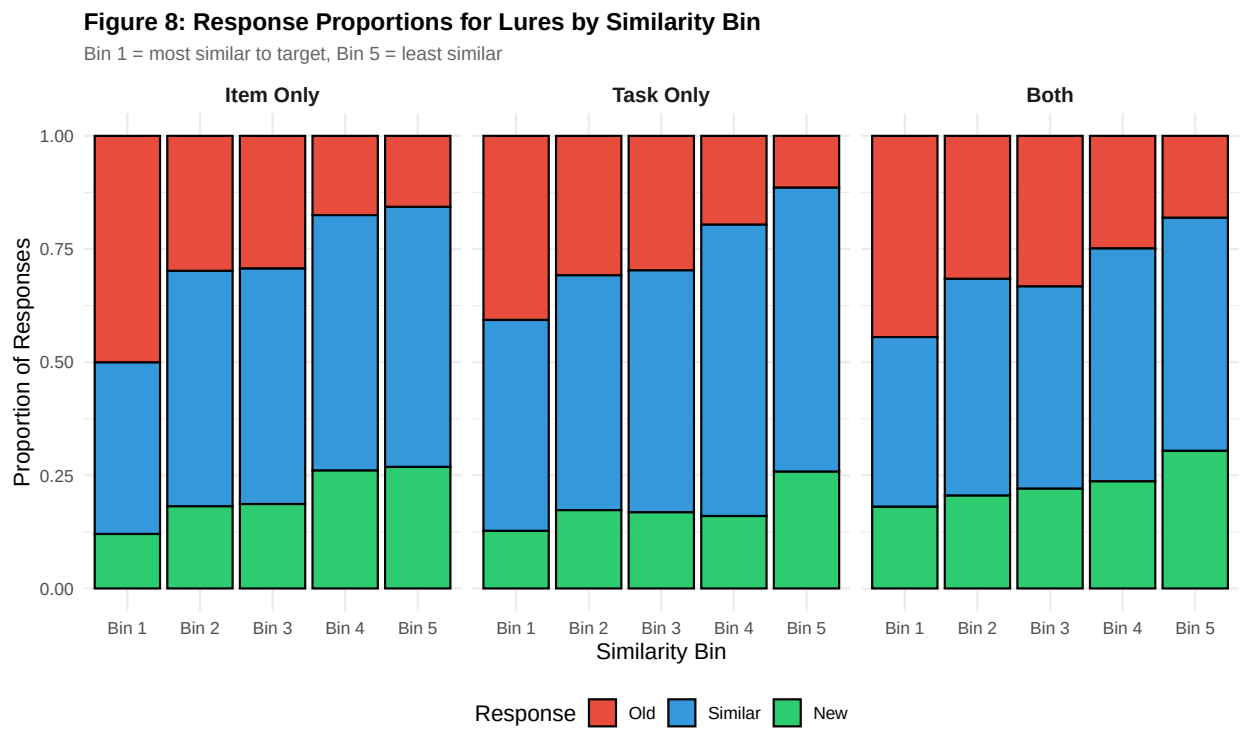
Table 4: Table 4: Lure Performance by Similarity Bin (1=most similar, 5=least similar)

condition	bin	n_trials	p_old	p_similar	p_new	mean_rt
Item Only	1	723	0.501	0.379	0.120	2.792
Item Only	2	667	0.298	0.520	0.181	3.055
Item Only	3	665	0.293	0.520	0.186	2.649
Item Only	4	668	0.175	0.564	0.260	2.687
Item Only	5	637	0.157	0.575	0.268	2.983
Task Only	1	354	0.407	0.466	0.127	2.152
Task Only	2	318	0.308	0.519	0.173	2.361
Task Only	3	333	0.297	0.535	0.168	2.255
Task Only	4	275	0.196	0.644	0.160	2.449
Task Only	5	349	0.115	0.628	0.258	2.510
Both	1	632	0.445	0.375	0.180	2.433
Both	2	595	0.316	0.479	0.205	2.426
Both	3	640	0.333	0.447	0.220	2.286
Both	4	651	0.249	0.515	0.237	2.298
Both	5	602	0.181	0.515	0.304	2.511

7.1 Figure 8: Lure Performance by Similarity Bin

```r
bin_props <- lure_with_bins %>%
  group_by(condition, bin, response) %>%
```

```r
  summarise(count = n(), .groups = "drop") %>%
  group_by(condition, bin) %>%
  mutate(proportion = count / sum(count)) %>%
  ungroup() %>%
  mutate(bin = factor(bin, labels = paste("Bin", 1:5)))

ggplot(bin_props, aes(x = bin, y = proportion, fill = response)) +
  geom_bar(stat = "identity", position = "stack", color = "black", size = 0.2) +
  facet_wrap(~ condition) +
  scale_fill_manual(values = c("Old" = "#E74C3C", "Similar" = "#3498DB", "New" = "#2ECC71"),
                    name = "Response") +
  labs(title = "Figure 8: Response Proportions for Lures by Similarity Bin",
       subtitle = "Bin 1 = most similar to target, Bin 5 = least similar",
       x = "Similarity Bin", y = "Proportion of Responses") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        plot.subtitle = element_text(size = 11, color = "grey40"),
        strip.text = element_text(face = "bold", size = 12),
        legend.position = "bottom")
```




```r
ggsave("figures/fig8_bin_performance.png", width = 10, height = 6, dpi = 300)
```

8. Lure Position Analysis (pre, mid, post)

```r
# Filter for lure items that have a specific position (pre, mid, post)
lure_pos_data <- all_test_data %>%
```

```r
  filter(stimulus_type == "Lure" & position_of_stimuli %in% c("pre", "mid", "post"))

# Compute performance by position
pos_performance <- lure_pos_data %>%
  group_by(condition, position_of_stimuli) %>%
  summarise(
    n_trials = n(),
    p_old = mean(response == "Old", na.rm = TRUE),
    p_similar = mean(response == "Similar", na.rm = TRUE),
    p_new = mean(response == "New", na.rm = TRUE),
    mean_rt = mean(rt, na.rm = TRUE),
    .groups = "drop"
  )

# Table
kable(pos_performance %>% mutate(across(where(is.numeric), ~ round(., 3))),
      caption = "Table 5: Lure Performance by Position of Stimuli (pre, mid, post)")
```

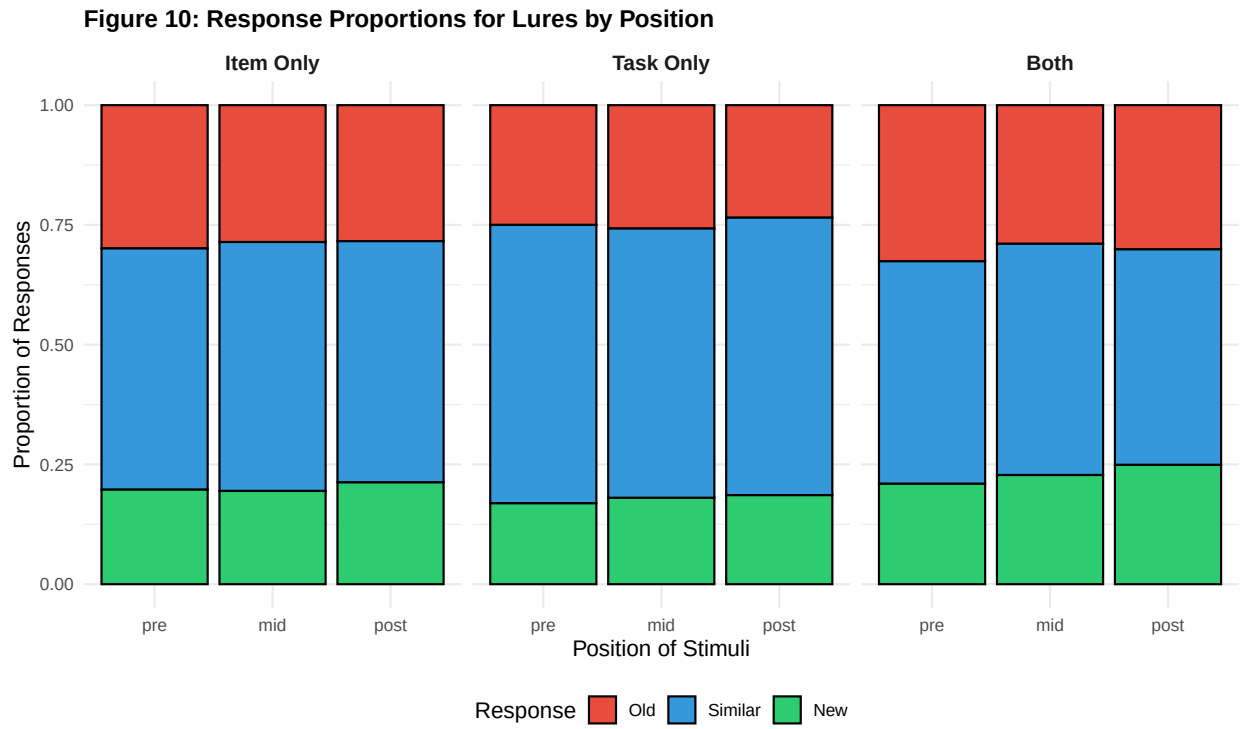
Table 5: Table 5: Lure Performance by Position of Stimuli (pre, mid, post)

condition	position_of_stimuli	n_trials	p_old	p_similar	p_new	mean_rt
Item Only	mid	1120	0.286	0.520	0.195	2.928
Item Only	post	1120	0.284	0.504	0.212	2.774
Item Only	pre	1120	0.299	0.504	0.197	2.791
Task Only	mid	1060	0.258	0.562	0.180	2.375
Task Only	post	1060	0.235	0.579	0.186	2.349
Task Only	pre	1060	0.250	0.581	0.169	2.289
Both	mid	1040	0.289	0.483	0.228	2.394
Both	post	1040	0.301	0.450	0.249	2.347
Both	pre	1040	0.326	0.464	0.210	2.423

8.1 Figure 10: Lure Performance by Position

```r
pos_props <- lure_pos_data %>%
  group_by(condition, position_of_stimuli, response) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(condition, position_of_stimuli) %>%
  mutate(proportion = count / sum(count)) %>%
  ungroup() %>%
  mutate(position_of_stimuli = factor(position_of_stimuli, levels = c("pre", "mid", "post")))

ggplot(pos_props, aes(x = position_of_stimuli, y = proportion, fill = response)) +
  geom_bar(stat = "identity", position = "stack", color = "black", size = 0.2) +
  facet_wrap(~ condition) +
  scale_fill_manual(values = c("Old" = "#E74C3C", "Similar" = "#3498DB", "New" = "#2ECC71"),
                    name = "Response") +
  labs(title = "Figure 10: Response Proportions for Lures by Position",
       x = "Position of Stimuli", y = "Proportion of Responses") +
  theme_minimal(base_size = 13) +
```

```
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 12),
        legend.position = "bottom")
```




```
ggsave("figures/fig10_pos_performance.png", width = 10, height = 6, dpi = 300)
```

9. Additional Descriptive Statistics

9.1 Overall Accuracy

```
# Compute overall accuracy: correct responses for each stimulus type
# Target → "Old", Lure → "Similar", Foil → "New"
all_test_data$correct <- with(all_test_data,
  (stimulus_type == "Target" & response == "Old") |
  (stimulus_type == "Lure" & response == "Similar") |
  (stimulus_type == "Foil" & response == "New")
)

accuracy_by_condition <- all_test_data %>%
  group_by(condition) %>%
  summarise(
    overall_accuracy = round(mean(correct, na.rm = TRUE), 3),
    target_accuracy = round(mean(correct[stimulus_type == "Target"], na.rm = TRUE), 3),
    lure_accuracy = round(mean(correct[stimulus_type == "Lure"], na.rm = TRUE), 3),
    foil_accuracy = round(mean(correct[stimulus_type == "Foil"], na.rm = TRUE), 3),
```

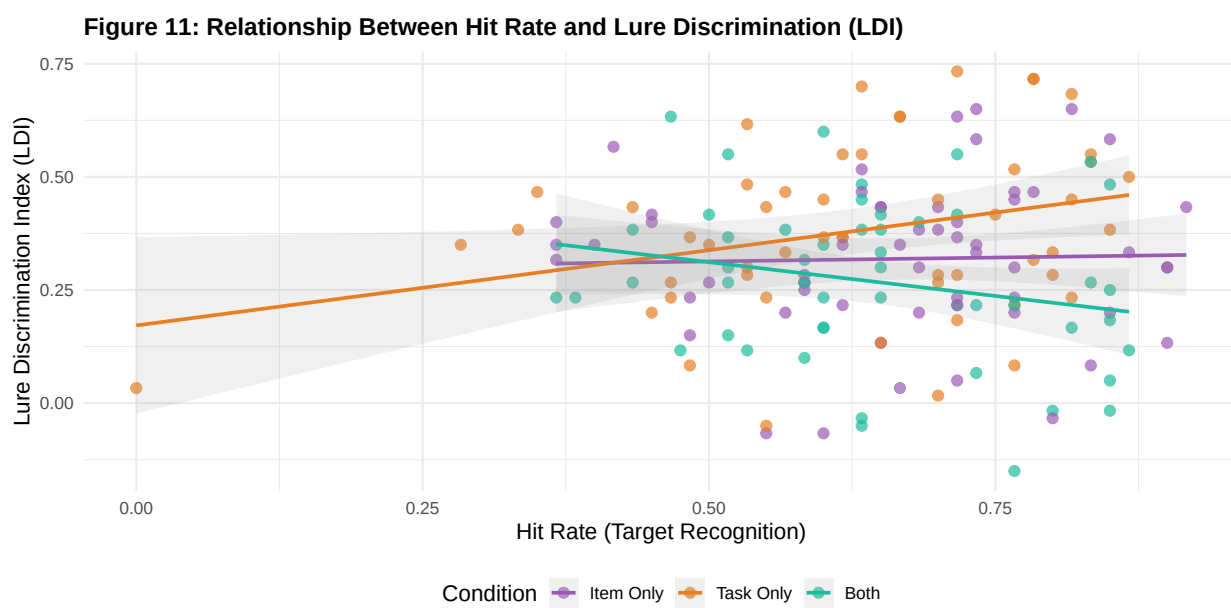
```r
    .groups = "drop"
  )

kable(accuracy_by_condition,
      caption = "Table 6: Accuracy Rates by Condition",
      col.names = c("Condition", "Overall", "Target", "Lure", "Foil"))
```

Table 6: Table 6: Accuracy Rates by Condition

Condition	Overall	Target	Lure	Foil
Item Only	0.626	0.669	0.509	0.775
Task Only	0.636	0.627	0.574	0.775
Both	0.592	0.638	0.466	0.754

9.2 Participant-Level Variability

```r
# Scatter plot: Hit Rate vs LDI colored by condition
ggplot(participant_metrics, aes(x = hit_rate, y = LDI, color = condition)) +
  geom_point(size = 2.5, alpha = 0.7) +
  geom_smooth(method = "lm", se = TRUE, alpha = 0.15, linewidth = 1) +
  scale_color_manual(values = c("Item Only" = "#9B59B6",
                                "Task Only" = "#E67E22",
                                "Both" = "#1ABC9C")) +
  labs(title = "Figure 11: Relationship Between Hit Rate and Lure Discrimination (LDI)",
       x = "Hit Rate (Target Recognition)", y = "Lure Discrimination Index (LDI)",
       color = "Condition") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14),
        legend.position = "bottom")
```



```r
ggsave("figures/fig11_hit_vs_ldi.png", width = 10, height = 5, dpi = 300)
```

10. Summary of Key Findings

```r
cat("=====================================================\n")
```

```
## =====================================================
```

```r
cat("       SUMMARY OF DESCRIPTIVE STATISTICS             \n")
```

```
##        SUMMARY OF DESCRIPTIVE STATISTICS
```

```r
cat("=====================================================\n\n")
```

```
## =====================================================
```

```r
for (cond in levels(participant_metrics$condition)) {
  d <- participant_metrics[participant_metrics$condition == cond, ]
  cat(paste0("--- ", cond, " Condition ---\n"))
  cat(paste0("  N participants: ", nrow(d), "\n"))
  cat(paste0("  Hit Rate: ", round(mean(d$hit_rate, na.rm=T), 3),
             " ± ", round(sd(d$hit_rate, na.rm=T), 3), "\n"))
  cat(paste0("  FA Rate:  ", round(mean(d$fa_rate, na.rm=T), 3),
             " ± ", round(sd(d$fa_rate, na.rm=T), 3), "\n"))
  cat(paste0("  LDI:      ", round(mean(d$LDI, na.rm=T), 3),
             " ± ", round(sd(d$LDI, na.rm=T), 3), "\n"))
  cat(paste0("  REC:      ", round(mean(d$REC, na.rm=T), 3),
             " ± ", round(sd(d$REC, na.rm=T), 3), "\n"))
  cat(paste0("  Mean RT:  ", round(mean(d$rt_overall, na.rm=T), 3),
             " ± ", round(sd(d$rt_overall, na.rm=T), 3), " sec\n\n"))
}
```

```
## --- Item Only Condition ---
##   N participants: 56
##   Hit Rate: 0.669 ± 0.145
##   FA Rate:  0.035 ± 0.059
##   LDI:      0.319 ± 0.169
##   REC:      0.634 ± 0.153
##   Mean RT:  2.669 ± 1.175 sec
##
## --- Task Only Condition ---
##   N participants: 53
##   Hit Rate: 0.627 ± 0.168
##   FA Rate:  0.031 ± 0.05
##   LDI:      0.381 ± 0.188
##   REC:      0.596 ± 0.174
##   Mean RT:  2.229 ± 0.567 sec
##
```

```
## --- Both Condition ---
##   N participants: 51
##   Hit Rate: 0.642 ± 0.133
##   FA Rate:  0.047 ± 0.071
##   LDI:      0.269 ± 0.175
##   REC:      0.594 ± 0.139
##   Mean RT:  2.32 ± 0.849 sec
```

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_India.utf8  LC_CTYPE=English_India.utf8
## [3] LC_MONETARY=English_India.utf8 LC_NUMERIC=C
## [5] LC_TIME=English_India.utf8
##
## time zone: Asia/Calcutta
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] knitr_1.51    tidyr_1.3.2   dplyr_1.1.4   ggplot2_4.0.1
##
## loaded via a namespace (and not attached):
##  [1] Matrix_1.7-4       gtable_0.3.6       compiler_4.5.2     tidyselect_1.2.1
##  [5] moments_0.14.1     splines_4.5.2      systemfonts_1.3.1  scales_1.4.0
##  [9] textshaping_1.0.4  yaml_2.3.12        fastmap_1.2.0      lattice_0.22-7
## [13] R6_2.6.1           labeling_0.4.3     generics_0.1.4     tibble_3.3.1
## [17] pillar_1.11.1      RColorBrewer_1.1-3 rlang_1.1.7        xfun_0.55
## [21] S7_0.2.1           cli_3.6.5          withr_3.0.2        magrittr_2.0.4
## [25] mgcv_1.9-3         digest_0.6.39      grid_4.5.2         rstudioapi_0.18.0
## [29] lifecycle_1.0.5    nlme_3.1-168       vctrs_0.7.0        evaluate_1.0.5
## [33] glue_1.8.0         farver_2.1.2       ragg_1.5.0         rmarkdown_2.30
## [37] purrr_1.2.1        tools_4.5.2        pkgconfig_2.0.3    htmltools_0.5.9
```